



SC3020 Database System Principles

Project 2 Report

Group 11

Code Link: [https://entuedu-](https://entuedu-my.sharepoint.com/:u:/g/personal/wang1847_e_ntu_edu_sg/ET_8zxxYmb9JiPEu3bTH7gcBRhHektQfNpa4Btwz5GIWeg?e=5ZBFu7)

[my.sharepoint.com/:u:/g/personal/wang1847_e_ntu_edu_sg/ET_8zxxYmb9JiPEu3bTH7gcBRhHektQfNpa4Btwz5GIWeg?e=5ZBFu7](https://entuedu-my.sharepoint.com/:u:/g/personal/wang1847_e_ntu_edu_sg/ET_8zxxYmb9JiPEu3bTH7gcBRhHektQfNpa4Btwz5GIWeg?e=5ZBFu7)

Name	Matric Number	Contribution
Wang Chuanze	U2240002G	Report
Izzat Hazim Bin Mustafa	U2222634G	Report
Isaac Lim Jia Le	U2321202C	Code
Janice Koh Swee En	U2222234H	Report
Ivan Soji Varghese	U2220650L	Code

College of Computing and Data Science

Nanyang Technological University

2024/2025 Semester 2

Introduction

SQL is a powerful tool for working with databases, but its structure can be hard to follow. Traditional SQL queries often require you to read them from the inside out, starting with the subqueries, which can be confusing. This project aims to solve that by converting SQL queries into a more straightforward, top-down format using PostgreSQL Query Execution Plans (QEPs). Our system automatically changes SQL queries into a step-by-step pipe-syntax format that reflects how the database actually runs them. Each step includes an estimated cost, helping users identify ways to improve performance. We also built a user-friendly interface with Streamlit, where users can enter SQL queries and instantly view both the pipe-syntax version and the original QEP.

This report includes:

- An overview of the system's architecture and its four main Python modules
- A description of the algorithm that transform QEPs into pipe syntax
- The design of the graphical interface and its key features
- Current limitations and ideas for future improvements

By using QEPs, our tool helps make SQL easier to understand while giving useful information about query performance, making it helpful for both new and experienced users.

Implementation

Our app consists of four files: `interface.py`, `pipesyntax.py`, `preprocessing.py`, and `project.py`.

`interface.py`:

- Creates a GUI with database connection settings, using Streamlit
- Provides areas for SQL input and pipe syntax output
- Shows both the pipe syntax and the original QEP
- Uses tabs to organize different views

`pipesyntax.py`:

- Converts QEP JSON to pipe syntax format
- Handles common PostgreSQL operations (scans, joins, aggregates, etc.)
- Recursively processes the plan tree
- Includes cost annotations as requested

`preprocessing.py`:

- Gets the query execution plan from PostgreSQL in JSON format
- Includes basic query validation

`project.py`:

- Main entry point that launches the GUI

Graphical User Interface (GUI) - `interface.py`

Purpose and Role

The `interface.py` is responsible for the GUI of the project. It creates a GUI with database connection settings using Streamlit and uses tabs to organise different views. This allows users to interact with the system intuitively. The code creates a GUI that enables the user to:

- Connect to a PostgreSQL database
- Input arbitrary SQL queries
- Retrieve the corresponding Query Execution Plan (QEP)
- View both the converted query and the raw QEP output

Database Connection

Host

localhost

Database

TPC-H

Username

postgres

Password



Connect

Disconnect

SQL to Pipe Syntax Converter

Convert your SQL queries to readable pipe syntax using PostgreSQL query execution plans.

SQL Query Input

Enter your SQL query:

```
SELECT
  nation,
  EXTRACT(YEAR FROM o_orderdate) AS o_year,
  SUM(l_extendedprice * (1 - l_discount) - ps_supplycost * l_quantity) AS profit
FROM
  part
JOIN partsupp ON p_partkey = ps_partkey
JOIN lineitem ON l_partkey = ps_partkey AND l_suppkey = ps_suppkey
```

[Convert to Pipe Syntax](#)

[Pipe Syntax Output](#) [Query Execution Plan](#)

Tools Used

```
import streamlit as st
import psycopg2
from psycopg2 import OperationalError
from psycopg2 import sql
from pipesyntax import generate_pipe_syntax
from preprocessing import get_query_plan
import time
```

- Streamlit: Chosen for its simplicity and fast deployment of interactive Python-based apps
- psycopg2: Used for connecting to and querying the PostgreSQL database

Function Overview

init_page

- Set up the visual layout and page metadata (wide, expanded)
- Displays the title and description

db_connection_sidebar

- Enables the user to connect to a PostgreSQL database instance by specifying:
 - Host
 - Database name
 - Username
 - Password
- Has both connect and disconnect functionality
- Displays success or failure status and auto-refresh the page accordingly

query_input_section

- User can input arbitrary SQL query in a text area
- Text is stored and later passed to the conversion logic

display_results - Displays the two main outputs in tabbed panels

- Pipe syntax output which is the converted SQL in pipe-syntax form
- The full QEP as returned from PostgreSQL

main

When pressing the "Convert to Pipe Syntax" button, the system will:

1. Get the QEP using the **get_query_plan** function from **preprocessing.py** (covered later).
2. Converts the QEP into pipe-syntax using the **generate_pipe_syntax** function from **pipesyntax.py** (covered later).
3. Display both outputs and do error handling if there is any database or converting failure.

Also, a rollback mechanism is included (**perform_rollback**) to reset transactions if there are database errors.

Design Rationale

The GUI was designed to have modularity as the UI logic is separated from the processing logic. It also maximises usability by using tabs.

Limitations

Streamlit is single-threaded and session-based. This will limit access to various users who are trying to access it at the same time. The error feedback currently only displays broad errors. Any specific errors like displaying specific QEP parsing issues will all give back a general error. Lastly, it assumes that the database is reachable and does not have advanced connection testing like SSL support and timeout control.

Pipe Syntax Generation - pipesyntax.py

Purpose and Role

The **pipesyntax.py** is the core transformation engine. The primary role is to convert a PostgreSQL QEP into a top-down pipe-syntax SQL query in natural language, which is then annotated with cost estimates derived from the QEP. This transformation helps the user better understand SQL logic as executed by the database engine.

Function Overview

generate_pipe_syntax

- Input: JSON query plan returned by PostgreSQL in string format
- Output: Multi-line string of the SQL logic in sequential pip-syntax format
- Error handling: Catch and report issues in parsing JSON or wrong/unexpected QEP formats

_process_plan

- Recursively traverse the tree structure of the QEP
- Each node (Seq Scan, Aggregate, etc.) is mapped to custom SQL-style operation
- Output lines are progressively appended with step description, cost annotation and optional conditions (filters, join conditions, etc.)

Key Features

Sequential Pipe-Syntax Generation

Each SQL operation logically flows from top to bottom which improves readability. The syntax supports indentation to reflect the hierarchy of nested plans.

Support for Major Node Types

The function handles:

- Seq Scan, Index Scan
- Hash Join, Nested Loop
- Aggregate, Sort, Limit

Estimated Cost Annotations

Each operation has a cost indication next to it to indicate the plan estimated cost. This helps with choosing which plan to go with for the best performance.

Database Connection

Host

Database

Username

Password

Deploy

Enter your SQL query:

```

SELECT
  SUM(l_extendedprice * l_discount) AS revenue
FROM
  lineitem
WHERE
  l_shipdate >= DATE '1994-01-01'
  AND l_shipdate < DATE '1995-01-01'
  AND l_discount BETWEEN 0.05 AND 0.07
          
```

Convert to Pipe Syntax

Pipe Syntax Output

Query Execution Plan

Converted Pipe Syntax

```

1  > AGGREGATE PLAIN [Cost: 170100.15]
2    > GATHER [Cost: 170100.13]
3      > AGGREGATE PLAIN [Cost: 169099.93]
4        FROM lineitem WHERE ((l_shipdate >= '1994-01-01'::date) AND (l_shipdate < '1995-01-01'::date) AND (l_discount >= 0.05) AND (l_discount < 0.07))
          
```

Conversion successful!

Query (simple complexity):

```

SELECT
  SUM(l_extendedprice * l_discount) AS revenue
FROM
  lineitem
WHERE
  l_shipdate >= DATE '1994-01-01'
  AND l_shipdate < DATE '1995-01-01'
  AND l_discount BETWEEN 0.05 AND 0.07
  
```

AND l_quantity < 24;

QEP:

► AGGREGATE PLAIN [Cost: 170100.15]

► GATHER [Cost: 170100.13]

► AGGREGATE PLAIN [Cost: 169099.93]

FROM lineitem WHERE ((l_shipdate >= '1994-01-01'::date) AND (l_shipdate < '1995-01-01'::date) AND (l_discount >= 0.05) AND (l_discount <= 0.07) AND (l_quantity < '24'::numeric))
[Cost: 168861.3]

Design Rationale

The recursive parsing allows natural traversal of PostgreSQL's nested QEP trees. The use of indentation and symbols (►) allow for easy readability of the output.

Limitations

Not all nodes are being handled. Less common nodes like CTE Scan or Bitmap Heap Scan are not supported and will use a generic formatting strategy.

Query Plan Extraction and Validation - preprocessing.py

Purpose and Role

The **preprocessing.py** acts as the interface between the app and the PostgreSQL database engine. It enables two operations:

1. Retrieving the QEP in JSON format.
2. Validating the correctness of the SQL queries before processing.

It supports the upstream logic required for the pipe-syntax generation and ensures it is robust by validating it early.

Function Overview

get_query_plan

- Executes the SQL query using PostgreSQL's [EXPLAIN \(FORMAT JSON\)](#) directive to retrieve the QEP.
- Returns the QEP as a formatted JSON string.
- Process:
 - Opens a cursor using the database connection.
 - Wraps the query in [EXPLAIN \(FORMAT JSON\)](#) to request the planner output.
 - Retrieves and serialises the returned QEP into a JSON string.

validate_query

- Performs a check using [EXPLAIN](#) to ensure the SQL query is valid.

- Returns:
 - True if the query executes successfully under [EXPLAIN](#)
 - False if a `psycopg2.Error` is thrown

Key Features

JSON-based QEP Retrieval

It uses PostgreSQL's support for JSON-formatted QEPs. This enables downstream traversal and structured transformation.

Formatted Output

The use of `json.dumps(..., indent=2)` ensures that the QEP is formatted in natural language when displayed in the interface.

Database Connection

Host: localhost

Database: TPC-H

Username: postgres

Password: [masked]

Connect Disconnect

Enter your SQL query:

```
o_orderdate >= DATE '1993-10-01'
AND o_orderdate < DATE '1994-01-01'
AND l_returnflag = 'R'
GROUP BY
  c_custkey, c_name
ORDER BY
  revenue DESC
LIMIT 20;
```

Convert to Pipe Syntax

Pipe Syntax Output Query Execution Plan

Converted Pipe Syntax

```
1 > LIMIT [Cost: 197193.09]
2 > ORDER BY (sum((lineitem.l_extendedprice * ('1'::numeric - lineitem.l_discount)))) DESC [Cost: 197328.55]
3 > AGGREGATE SORTED GROUP BY customer.c_custkey [Cost: 195758.77]
4 > GATHER MERGE [Cost: 194734.5]
5 > AGGREGATE SORTED GROUP BY customer.c_custkey [Cost: 188528.97]
6 > ORDER BY customer.c_custkey [Cost: 188812.83]
7 > INNER JOIN ON orders.o_custkey = customer.c_custkey [Cost: 186323.21]
8 > INNER JOIN ON lineitem.l_orderkey = orders.o_orderkey [Cost: 181257.67]
9 > FROM lineitem WHERE (l_returnflag = 'R')::bpchar [Cost: 143856.28]
10 > HASH [Cost: 35511.0]
11 > FROM orders WHERE ((o_orderdate >= '1993-10-01'::date) AND (o_orderdate < '1994-01-01'::date)) [Cost: 35511.0]
12 > HASH [Cost: 4225.0]
13 > FROM customer [Cost: 4225.0]
```

Query (medium complexity):

```
SELECT
  c_custkey,
  c_name,
  SUM(l_extendedprice * (1 - l_discount)) AS revenue
FROM
  customer
JOIN orders ON c_custkey = o_custkey
JOIN line item ON o_orderkey = l_orderkey
WHERE
  o_orderdate >= DATE '1993-10-01'
  AND o_orderdate < DATE '1994-01-01'
  AND l_returnflag = 'R'
GROUP BY
  c_custkey, c_name
ORDER BY
  revenue DESC
```

LIMIT 20;

QEP:

- LIMIT [Cost: 197193.09]
 - ORDER BY (sum((lineitem.l_extendedprice * ('1'::numeric - lineitem.l_discount))))
DESC [Cost: 197328.55]
 - AGGREGATE SORTED GROUP BY customer.c_custkey [Cost: 195750.77]
 - GATHER MERGE [Cost: 194734.5]
 - AGGREGATE SORTED GROUP BY customer.c_custkey [Cost: 188520.97]
 - ORDER BY customer.c_custkey [Cost: 188012.83]
 - INNER JOIN ON orders.o_custkey = customer.c_custkey [Cost: 186323.21]
 - INNER JOIN ON lineitem.l_orderkey = orders.o_orderkey [Cost: 181257.67]
FROM lineitem WHERE (l_returnflag = 'R'::bpchar) [Cost: 143856.28]
 - HASH [Cost: 35511.0]
FROM orders WHERE ((o_orderdate >= '1993-10-01'::date) AND (o_orderdate < '1994-01-01'::date)) [Cost: 35511.0]
 - HASH [Cost: 4225.0]
FROM customer [Cost: 4225.0]

Database Connection

Host
localhost

Database
TPC-H

Username
postgres

Password

Connect

Disconnect

ORDER BY
nation, o_year DESC;

Convert to Pipe Syntax

Pipe Syntax Output Query Execution Plan

Converted Pipe Syntax

```
1 ➤ AGGREGATE SORTED GROUP BY nation.*, (EXTRACT(year FROM orders.o_orderdate)) [Cost: 151312.5]
2 ➤ ORDER BY nation.*, (EXTRACT(year FROM orders.o_orderdate)) DESC [Cost: 151312.47]
3 ➤ INNER NESTED LOOP JOIN [Cost: 151312.46]
4 ➤ GATHER [Cost: 151311.98]
5 ➤ INNER NESTED LOOP JOIN ON (part.p_partkey = partsupp.ps_partkey) [Cost: 150311.88]
6 ➤ INNER JOIN ON lineitem.l_suppkey = supplier.s_suppkey [Cost: 150101.96]
7 ➤ INNER JOIN ON lineitem.l_partkey = part.p_partkey [Cost: 149370.21]
8 FROM lineitem [Cost: 137605.02]
9 ➤ HASH [Cost: 5169.67]
10 FROM part WHERE ((p_name)::text ~ '%green%'::text) [Cost: 5169.67]
11 ➤ HASH [Cost: 442.4]
12 ➤ INNER JOIN ON supplier.s_nationkey = nation.n_nationkey [Cost: 442.4]
13 FROM supplier [Cost: 380.0]
14 ➤ HASH [Cost: 24.29]
15 ➤ INNER JOIN ON nation.n_regionkey = region.r_regionkey [Cost: 24.29]
16 FROM nation [Cost: 11.7]
17 ➤ HASH [Cost: 12.12]
18 FROM region WHERE (r_name = 'AMERICA'::bpchar) [Cost: 12.12]
19 FROM partsupp USING INDEX partsupp_pkey WHERE ((ps_partkey = lineitem.l_partkey) AND (ps_suppkey = lineitem.l_suppkey)) [Cost: 0.46]
20 FROM orders USING INDEX orders_pkey WHERE (o_orderkey = lineitem.l_orderkey) [Cost: 0.47]
```

Query (difficult complexity):

```
SELECT
    nation,
    EXTRACT(YEAR FROM o_orderdate) AS o_year,
    SUM(l_extendedprice * (1 - l_discount) - ps_supplycost * l_quantity) AS profit
FROM
    part
```

```

JOIN partsupp ON p_partkey = ps_partkey
JOIN lineitem ON l_partkey = ps_partkey AND l_suppkey = ps_suppkey
JOIN supplier ON s_suppkey = l_suppkey
JOIN orders ON l_orderkey = o_orderkey
JOIN nation ON s_nationkey = n_nationkey
JOIN region ON n_regionkey = r_regionkey
WHERE
    p_name LIKE '%green%'
    AND r_name = 'AMERICA'
    AND o_orderdate BETWEEN DATE '1995-01-01' AND DATE '1996-12-31'
GROUP BY
    nation, o_year
ORDER BY
    nation, o_year DESC;

```

QEP:

```

► AGGREGATE SORTED GROUP BY nation.*, (EXTRACT(year FROM
orders.o_orderdate)) [Cost: 151312.5]
    ► ORDER BY nation.*, (EXTRACT(year FROM orders.o_orderdate)) DESC [Cost:
151312.47]
        ► INNER NESTED LOOP JOIN [Cost: 151312.46]
            ► GATHER [Cost: 151311.98]
                ► INNER NESTED LOOP JOIN ON (part.p_partkey = partsupp.ps_partkey) [Cost:
150311.88]
                    ► INNER JOIN ON lineitem.l_suppkey = supplier.s_suppkey [Cost: 150101.96]
                        ► INNER JOIN ON lineitem.l_partkey = part.p_partkey [Cost: 149370.21]
                            FROM lineitem [Cost: 137605.02]
                                ► HASH [Cost: 5169.67]
                                    FROM part WHERE ((p_name)::text ~~ '%green%':text) [Cost: 5169.67]
                                ► HASH [Cost: 442.4]
                                    ► INNER JOIN ON supplier.s_nationkey = nation.n_nationkey [Cost: 442.4]
                                        FROM supplier [Cost: 380.0]
                                            ► HASH [Cost: 24.29]
                                                ► INNER JOIN ON nation.n_regionkey = region.r_regionkey [Cost: 24.29]
                                                    FROM nation [Cost: 11.7]
                                                        ► HASH [Cost: 12.12]
                                                            FROM region WHERE (r_name = 'AMERICA':bpchar) [Cost: 12.12]
                                                        FROM partsupp USING INDEX partsupp_pkey WHERE ((ps_partkey = lineitem.l_partkey)
AND (ps_suppkey = lineitem.l_suppkey)) [Cost: 0.46]
                                                            FROM orders USING INDEX orders_pkey WHERE (o_orderkey = lineitem.l_orderkey) [Cost:
0.47]

```

Design Rationale

The early validation allows the interface layer to reject any wrong or malformed queries which improves the user experience and reduces back-end errors. It also provides foundational input required by **pipesyntax.py** for converting the QEP to pipe syntax.

Limitations

It does not evaluate query semantics. Only syntax is validated. Semantic issues like invalid logic or data type may still pass **validate_query** but fail during execution. It also does not sanitise user input and just executes raw queries.

Application Entry Point - **project.py**

Purpose and Role

The **project.py** is the main entry point of the system. Its only purpose is to initialise and launch the app by invoking the GUI defined in **interface.py**.

Overview

from interface import main

This imports the **main()** function from **interface.py** which sets up the entire GUI app. This includes the database connection management, query input, pipe-syntax generation and QEP visualisation.

Conclusion

SQL remains the standard for database interactions, yet its inverted logic continues to challenge both new and experienced users. This work explores an alternative approach by reconstructing queries to reflect their actual execution flow through pipe syntax representation.

The developed system demonstrates:

- How execution plans can serve as frameworks for transformation
- The significance of showing queries step-by-step with cost information for better performance understanding
- The application of query optimization techniques in practical database environments

This approach can be expanded to handle more SQL commands and database engines, while incorporating deeper performance analysis. The method also shows promise for education use, helping learners visualise query execution, and as a debugging tool for database professionals.

Rather than presenting a finished solution, this project opens pathways for rethinking how we interact with and understand SQL queries. The results suggest promising directions for making database technologies more accessible without sacrificing their power.